

UNIDADE 1

Conceptos Básicos e preparación do entorno de desenvolvemento en VUE

Obxectivos da unidade

- Comprender os conceptos fundamentais de **Vue.js** e o seu enfoque baseado en **compoñentes** e **reactividade**.
- Analizar a estrutura básica dun proxecto creado con **Vite e Vue 3**.
- Configurar correctamente o **entorno de desenvolvemento** en Visual Studio Code con extensións útiles.
- Crear e executar o **primeiro proxecto funcional en Vue**, comprendendo o papel dos ficheiros principais (*App.vue*, *main.js*, *HelloWorld.vue*).

A.1. Conceptos Básicos

- Creado por **Evan You**, ex-empleado de Google onde traballaba con Angular.
- **Framework progresivo**, para construír **interfaces de usuario** en contornas Web.
- Facilita a creación de **aplicacións reactivas**, e baseadas en **compoñentes**.
- As aplicacións Web son preferentemente do tipo **SPA** (*Single Page Application*).
- **Vue.js** inspírase o **patrón Modelo-Vista-ViewModel (MVVMA)** separando a **lóxica da interface de usuario (vista)** da **lóxica da aplicación (modelo e view-model)**.
- **Escalable**, a partires da **integración de novos compoñentes** según a necesidade da aplicación.
- **Lixeiro**, e con unha curva de aprendizaxe suave.

Vantaxes	Inconvenientes
Excelente documentación e curva de aprendizaxe suave.	Comunidade máis pequena que React ou Angular.
Reactividade automática dos datos.	Ecosistema menos maduro, aínda que sólido.
Flexibilidade para proxectos pequenos e complexos	En proxectos grandes pode precisar complementos adicionais.
Rendemento na execución da aplicación	Cantidade de <i>plugins</i> menor fronte a <i>React</i> ou <i>Angular</i> .

Conceptos básicos a ter en conta:

- **Aplicacións reactivas:** actualizan automaticamente a súa interface de usuario cando cambian os datos subxacentes, **sen que o usuario teña que refrescar** a páxina ou facer accións adicionais.
- **SPA:** **só se carga unha vez a páxina** HTML, e despois o contido cambia dinamicamente sen recargar a *web*.

Ambos conceptos están interrelacionados.

Por outra banda, Aínda que *React*, de momento, domina no mercado de *frameworks* no lado do *frontend*, especialmente en empresas grandes, a pregunta que xorde una pregunta lóxica: **¿por qué ensinar Vue ó meu alumnado?**

A resposta está no **enfoque didáctico**. **Vue destaca** pola súa **sintaxe máis clara** e a **separación natural** entre **template, lóxica e estilos**. Isto facilita que o alumnado comprenda desde o inicio como se estrutura unha aplicación frontend moderna.

Iniciarse directamente co *React* pode xerar certa frustración porque introduce **demasiadas conceptos á vez**: funcións que son compoñentes, un elevado nivel de abstracción inicial, estados inmutables, mezcla HTML dentro de JS...

Esta complexidade adoita levar a que o alumnado **memorice patróns sen chegar a razoalos**, ao non existir unha estrutura clara onde distinguir que parte corresponde á vista, á lóxica ou ao estado.

Todo esto evítase con **Vue**. O alumno traballa cunha estrutura clara dun proxecto dende o principio: **"aquí vai o que se ve, aquí o que fai e eiquí como se ve"**. Isto permite consolidar conceptos fundamentais do *frontend* co que o salto a *React* é moi máis doado.

Na súa defensa, en canto ao mercado laboral, *React* domina arredor de dous terzos das ofertas, mentres que o terzo restante repártese principalmente entre *Angular* e **Vue**. Por iso, **Vue** non substitúe *React* na formación, senón que actúa como unha base sólida que facilita a aprendizaxe posterior doutros *frameworks* máis complexos. Aínda así, **Vue** si

está a crecer especialmente en pequenas empresas e startups fortalecendo o seu ecosistema.

A.2 Estrutura básica dun proxecto VUE.

Una vez iniciado un proxecto **Vue**, a estrutura xeral do mesmo é a que se mostra a continuación.

- nomeproxectovue/	
└ node_modules/	# dependencias e librerías instaladas mediante npm o yarn
└ public/	# arquivos públicos (favicon, index.html base, ...). Non pasa polo build
└ favicon.ico	
└ src/	# código fonte da app, o que realmente se compila (build)
└ assets/	# recursos estáticos (imaxes, videos, estilos globais, fontes...)
└ components/	# compoñentes vue (.vue) reusablees
└ views/	# vista e páxinas completas se se usa Vue Router
└ router/	# configuración de Vue Router (rutas)
└ store/	# xestión do estado global (Vuex ou Pinia)
└ App.vue	# compoñente raíz, pode ir fora do src
└ main.js	# punto entrada aplicación
└ .gitignore	# arquivos ou directorios a ignorar en Git
└ package.json	# dependencias, scripts e metadatos do proxecto
└ package-lock.json	# versión exacta das dependencias (pode ser yarn.lock)
└ vite.config.js	# configuración de Vite (a versión máis moderna de Vue)
└ README.MD	# información e documentación do proxecto

Dita estrutura **non é fixa** pode variar en función das necesidades e evolución do proxecto. Tampouco son necesarias todas nun proxecto pequeno, e nos grandes podemos necesitar engadir directorios ou arquivos dos que se mostran na imaxe. Por exemplo, se temos nunha aplicación con *CRUD*, o habitual é engadir arquivos de consultas, actualizacións... nun arquivo *js*. E se son moitos, *clientes.js*, *produtos.js*..., gardalos nun directorio, xeralmente chamado **api**.

A.3. ¿Que é un compoñente?

É unha **unidade independente y reutilizable** da interface de usuario o que facilita o seu mantemento.

Encapsula, repectivamente, **HTML, JavaScript e CSS** para que traballen xuntos.

O QUE SE VE,

O QUE FAI

COMO SE VE.

Poden comunicarse con outros a través de **props** (recibir datos) e **eventos** (enviar datos de volta) ou con estados globais creados con **Pinia**.

Ten tres partes: **template**, **script (setup)** e **style (scope)**

<!-- parte del template, a vista, O QUE SE VE -->

```
<template>
  <button @click="contador++">Clicks: {{ contador }}</button>
</template>
```

<!-- zona de scripts O QUE FAI -->

```
<script setup>
  export default {
    data() {
      return { contador: 0 }
    }
  }
</script>
```

<!-- zona de estilos COMO SE VE-->

```
<style scope> <!-- scope: o estilo só se aplica ao compoñente en cuestión -->
  button { background: lightblue; }
</style>
```

Un aspecto importante nas últimas versión de Vue foi a inclusión no script do termo **setup**.

En *Vue 2*, a lóxica dun compoñente, estaba espallada:

- *data*
- *methods*
- *computed*
- *watch*
- *mounted*

Funcionaba ben pero cando o proxecto crecía o mantemento facíase complicado. Con *Vue 3*, **setup()** é o punto de entrada da lóxica dun compoñente. Ademais, evitase o uso de claves como *export default*, *returns*, que aínda que funcionaba ben, pero cando o proxecto escalaba o mantemento facíase complicado e confuso.

Setup é o **primeiro que se executa antes de que o compoñente se mostre** e onde se define todo o estado reactivo e a lóxica. Todo o que declares no **setup()** (ou uses con *script setup*) está dispoñible no *template*.

En resumo, **unifica toda a lóxica**. En vez de ter, datos por un lado, métodos por outro e *computed* noutro, agora temos **todo xunto**, o que facilita a reusabilidade das funcións, todo está máis organizado por funcionalidade e, o máis importante, existe unha maior lexibilidade. Hai outras vantaxes engadidas pero estas son a base en *Vue 3*.

A.4. Preparación da contorna de desenvolvemento

Imos a establecer previamente unha serie de consideracións:

- Sistema Operativo será **Ubuntu 24.04** ou ben **Windows 11**, non hai diferenzas substanciais, agás o *case-sensitive* de Linux as maiúsculas e minúsculas e, moi rara vez, a versión de algunha librería. Irei saltando alternativamente entre os dous para que vexades que non hai problema.
- O Entorno de Desenvolvemento será **Visual Code** incluíndo algunhas extensións que se indicarán para facilitar a codificación do código.
- Usaremos a **aprendizaxe baseada en proxectos**. A partires dun sinxelo suposto práctico iremos engadindo funcionalidades no mesmo.
- Este curso é unha introdución ao *framework* onde veremos, ademais da sintaxe e módulos básicos dun compoñente, o enrutamento e operacións *CRUD* sinxelas. Nun curso máis avanzado quedarían, principalmente, temas como a autenticación, uso de librerías avanzadas como sistemas de pago ou o *build* da aplicación, entre outros.
- O obxectivo é que ao rematar teñamos un pequeno proxecto funcional e os coñecementos necesarios para seguir avanzando.

Unha vez establecido o contexto empecemos.

Paso 1. Instalación de Visual Code e as extensións máis usada.

En **Ubuntu** podemos facelo dende a **liña de comandos**:

```
$ sudo apt update && sudo apt upgrade -y
```

```
$ sudo apt install software-properties-common apt-transport-https wget -y
```

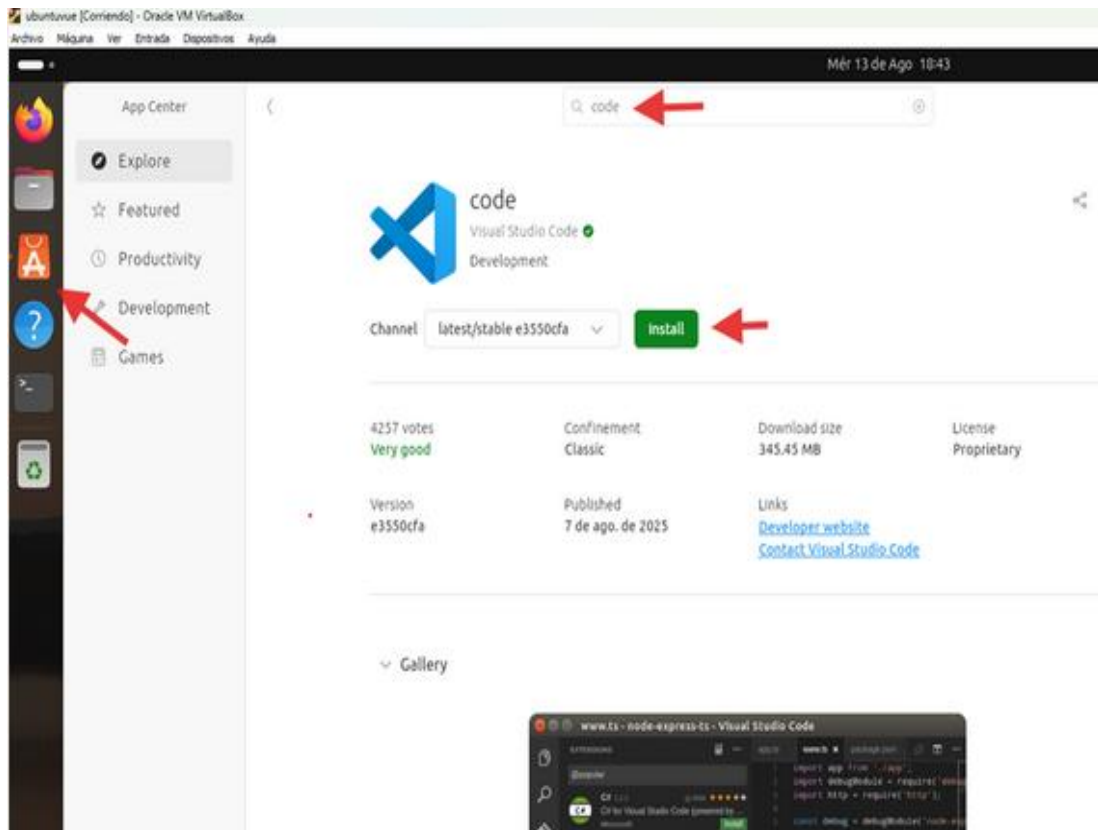
```
$ wget -q https://packages.microsoft.com/keys/microsoft.asc -O- | sudo apt-key add -
```

```
$ sudo add-apt-repository "deb [arch=amd64] https://packages.microsoft.com/repos/code stable main"
```

```
$ sudo apt update && sudo apt install code -y
```

```
$ code --version    (verifica a versión)
```

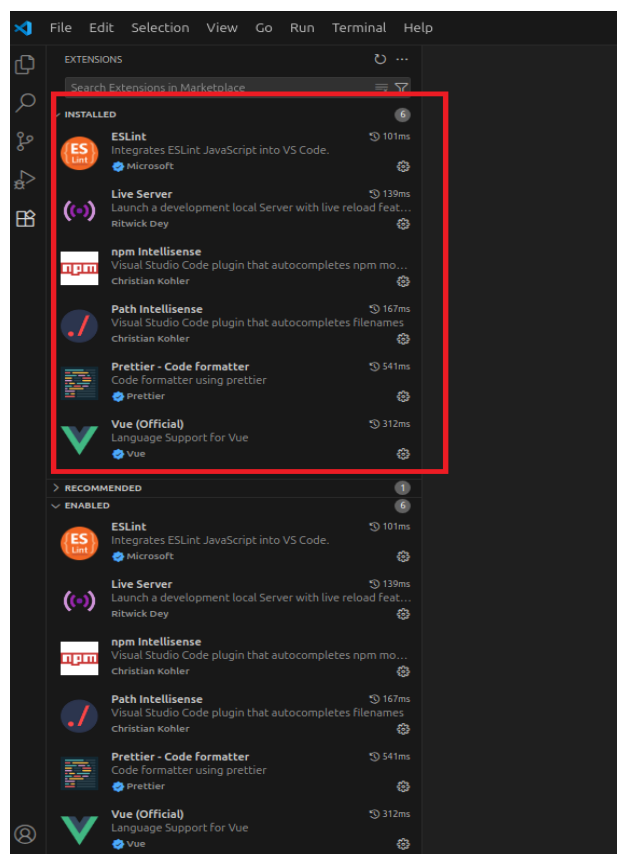
Ou ben dende a APP de Ubuntu e non nos complicamos.



Paso 2. Instalación das extensións máis utilizadas en VS Code para a codificación con **VUE**.

Aínda que non son estritamente necesarias, facilitan a codificación traballando algunhas delas de forma transparente ao usuario.

Dado por feito que sabemos instalar extensións en VSCode o resultado final quedaría así. Senón podemos consultar como instalar extensión **eiquí**



Vue Language Features (Volar). Reemplaza a Vetur (eliminar se aparece) en Vue 3. Autocompletado, resaltado de sintaxes, IntelliSense	ID: <i>Vue.volar</i>
ESLint. Mantén o código consistente y libre de erros comúns. Require configurar o arquivo .eslintrc no proxecto	ID: <i>dbaeumer.vscode-eslint</i>
Prettier - Code formatter. Formatea automáticamente o código cun estilo unificado. Usalo como formateador por defecto.	ID: <i>esbenp.prettier-vscode</i>
npm Intellisense. Autocompletado moi útil para imports de módulos instalados vía npm.	ID: <i>christian-kohler.npm-intellisense</i>
Path Intellisense. Autocompletado para rutas de arquivos, evita problemas. Complementa a anterior.	ID: <i>christian-kohler.path-intellisense</i>
TypeScript Vue Plugin (Volar) (opcional se se usa TS). Mellora soporte de TypeScript en Vue.	ID: <i>Vue-Official</i>

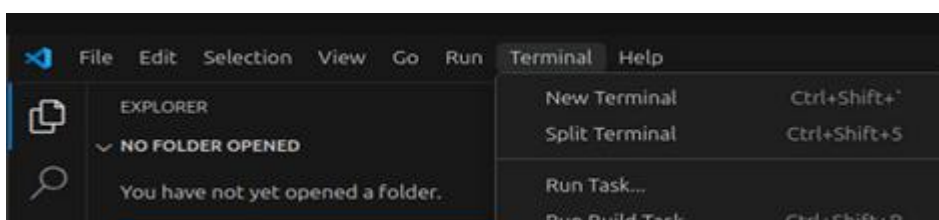
Dado por feito que sabemos instalar extensións en VSCode o resultado final quedaría así. Senón podemos consultar como instalar extensión [eiquí](#)

Tamén podemos engadir **Live Server (Ritwick Dey)** se se quere mostrar exemplos rápidos de HTML/JS fora de Vue. Persoalmente prefiro evitalo xa que en proxectos máis complexos no rende como quero e utilizo os navegadores.

Paso 3. Creamos o proxecto Vue que o dividiremos en tres etapas:

3.1. Situámonos no **terminal de comandos de Visual Code**, no directorio que consideres oportuno. Cando creamos o proxecto vue, o *framework* creará o directorio do mesmo onde estarán todos os arquivos necesarios que vimos na imaxe da páxina 5.

Terminal → New Terminal



3.2. Agora temos que instalar Node.js: **npm** ou **node package manager** ou ben **nvm** ou **node versión manager** (recomendado para Linux).

En **Windows**, descarga dende esta [ligazón](#). Non descarga *nvm*, só *npm*. Para o proxecto básico é suficiente. En proxectos máis grandes, proxectos en equipo ou proxectos que quedaron desactualizados si é necesario.

En **Linux** temos **dúas opcións**:

- **npm**: esta ben para empezar pero non instala o máis recente e pode afectar algún paquete.

<i>\$ sudo apt update</i>
<i>\$ sudo apt install nodejs npm -y</i>

- **nvm (Node version manager)**: a instalación é máis complexa, tampouco tanto, pero a máis recente e facilita moito o control das versións dos paquetes. Se estás **en Linux é a recomendada**.

Descargamos o paquete de instalación. Pode ser necesario instalar **curl**, as veces Ubuntu pode non traelo por defecto.

<i>\$sudo apt install curl</i>
<i>\$curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh sh</i>
<i>\$ source ~/.bashrc</i>

Recargamos a configuración



```

• usuario@cursovue:~$ source ~/.bashrc
○ usuario@cursovue:~$

```

```

• usuario@cursovue:~$ curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
=> Downloading nvm as script to '/home/usuario/.nvm'

=> Appending nvm source string to /home/usuario/.bashrc
=> Appending bash completion source string to /home/usuario/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"

```

E procedemos a instalación da última versión –LTS de Node.js e comprobamos as versións.

<code>\$ nvm install --lts</code>	<pre> usuario@cursovue:~\$ nvm install --lts Installing latest LTS version. Downloading and installing node v22.18.0... Downloading https://nodejs.org/dist/v22.18.0/node-v22.18.0-linux-x64.tar.xz... </pre>	
<code>\$ nvm use --lts</code>	<pre> • usuario@cursovue:~\$ nvm use --lts #usar la última versión Now using node v22.18.0 (npm v10.9.3) • usuario@cursovue:~\$ node -v #mostar versión de Node.js v22.18.0 • usuario@cursovue:~\$ npm -v #comprobar que funciona 10.9.3 • usuario@cursovue:~\$ </pre>	

3.3. Finalmente tanto en Linux como en Windows, **lanzamos un proxecto Vue**.

No meu caso chamarei **proxectovue** dende o terminal de Visual Code, no directorio que consideres oportuno.

```

• usuario@cursovue:~$ npm create vite@latest proxectovue -- --template vue
Need to install the following packages:
create-vite@7.1.1
Ok to proceed? (y) y

> npx
> create-vite proxectovue --template vue

◇ Scaffolding project in /home/usuario/proxectovue...

Done. Now run:

cd proxectovue
npm install
npm run dev

npm notice

```

<code>\$ npm create vite@latest proxectovue --template vue</code>	LINUX
<code>C:/users> npm create vite@latest proxectovue --template vue</code>	WINDOWS

Pode suceder que nos indique que *framework* queremos usar, eleximos **Vue**.

Logo eliximos **JS** como *variant*, e contestamos que **non** a versión beta de **Vite**, aínda non estable. Un apunte, a versión beta de **Vite** contén como melloras un novo **rolldown**, que mellora a compilación de producións, sendo máis rápidas, unifica certas ferramentas que se usan tanto en *dev* como en produción e certas opcións avanzadas de *Typescript*, entre outras.

```

◇ Select a framework:
  Vue

◇ Select a variant:
  JavaScript

◇ Use Vite 8 beta (Experimental)?:
  No

◇ Install with npm and start now?
  Yes

◇ Scaffolding project in C:\Users\jcarl\proxectovue...

◇ Installing dependencies with npm...
npm warn Unknown env config "template". This will stop working in the next major version of npm.

```

Algunhas consideracións adicionais sobre as imaxes anteriores:

O xestor de paquetes crea a carpeta do proxecto.	proxectovue
Usará a versión máis recente de <i>Vite</i>	<code>-latest</code>
Usará a plantilla básica de <i>Vue</i> .	<code>--template</code>
Entramos na carpeta do proxecto.	<code>cd proxectovue</code>
Instalamos dependencias e/ou actualizamos	<code>npm install npm update</code>
Lanzamos o proxecto	<code>npm run dev</code>

Outra mellora é que, se nos fixamos, tras a instalación o *framework* **arrinca automaticamente**.

```
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Se non é así, seguimos lendo.

Tamén pode presentar algúns **warnings** como é a actualización de **npm** indicando a forma de facelo. Neste curso de introdución pódese omitir.

Imos finalizar a instalación cos pasos que se indican, entrando no directorio do proxecto, **instalando as dependencias e lanzando o proxecto**.

```

usuario@cursovvue:~$ cd proxectovue/
usuario@cursovvue:~/proxectovue$ npm install

added 34 packages, and audited 35 packages in 1m

6 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
  
```

⇒

```

usuario@cursovvue:~/proxectovue$ npm run dev
> proxectovue@0.0.0 dev
> vite

VITE v7.1.3 ready in 1428 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
  
```

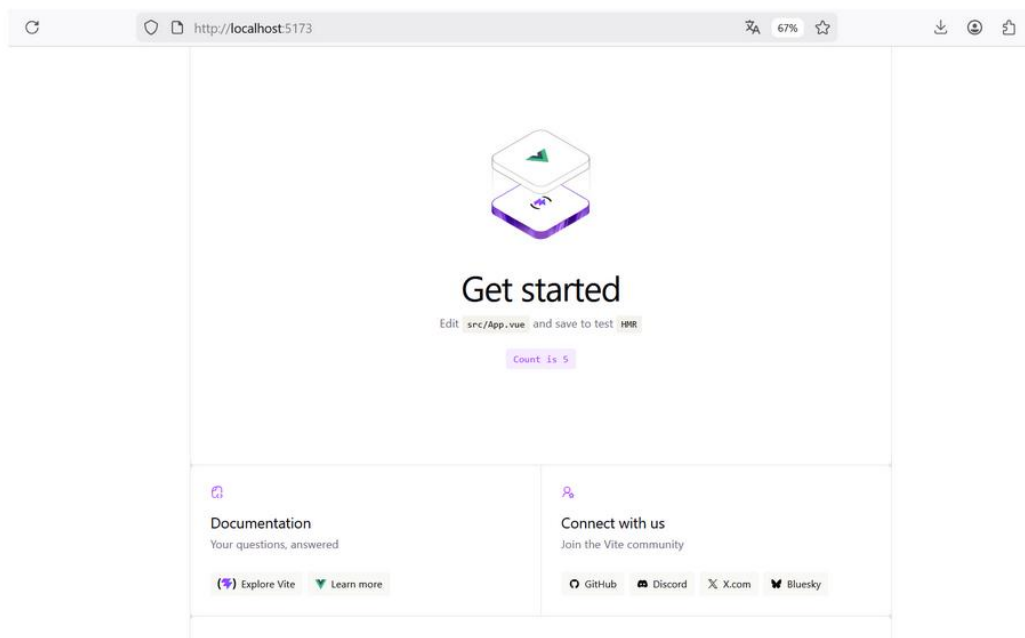
Una vez temos o proxecto en marcha queda falar de dous comando habituais que son **npm install** e **npm update**.

- **npm install**: instala ou verifica as dependencias segundo o que está especificado no *package.json*. Se non están instaladas, instalará; se xa están, asegúrase de que están correctas.
- **npm update**: actualiza as dependencias que xa están instaladas ás versións máis recentes que coincidan coas restricións de versión no *package.json*. Por exemplo, se no *package.json* pónse que o paquete *lodash* debe ser *^4.17.0*, *npm update* só actualizará a versión a algo dentro da gama 4.x.x, pero non á versión 5.x.x. Se non tén *^* entón non se actualiza.

Recordemos que *package.json* contén información sobre o proxecto, as dependencias que utiliza e como debe executarse ou compilarse.

```
package.json M X
} package.json > ...
1  {
2    "name": "proxectovue",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "preview": "vite preview"
10   },
11   "dependencies": {
12     "vue": "^3.5.18",
13     "vue-router": "^4.6.3"
14   },
15   "devDependencies": {
16     "@vitejs/plugin-vue": "^6.0.1",
17     "vite": "^7.1.2"
18   }
19 }
20
```

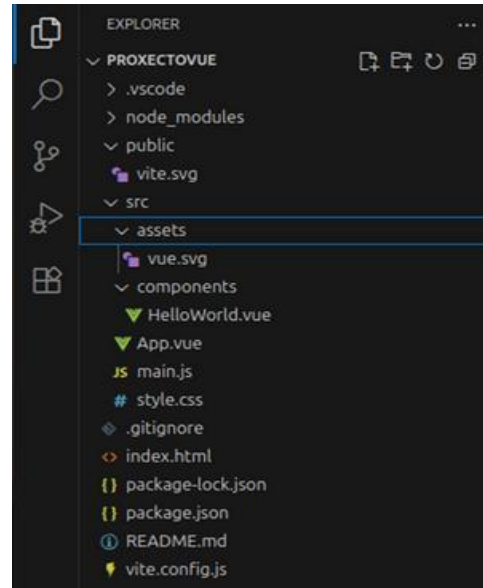
Só temos que cliquesar na ligazón e no navegador apareceranos a páxina inicial.



Por outra banda, se abrimos **Open Folder** en Visual Code aparécenos algo que nos resulta familiar que é a estrutura básica dun proxecto Vue que analizamos no apartado A.2.

Como xa indicamos antes en desenvolvemento pódese seguir esta distribución pero en produción separamos o *backend* do *frontend*.

Imos analizar **arquivo** *HelloWorld.vue*.



```

HelloWorld.vue
src > components > HelloWorld.vue > {} script setup
1  <template>
2    <h1>{{ msg }}</h1>
3
4    <div class="card">
5      <button type="button" @click="count++>count is {{ count }}</button>
6      <p>
7        Edit
8        <code>components/HelloWorld.vue</code> to test HMR
9      </p>
10   </div>
11
12   <p>
13     Check out
14     <a href="https://vuejs.org/guide/quick-start.html#local" target="_blank">
15       >create-vue</a>
16     >, the official Vue + Vite starter
17   </p>
18   <p>
19     Learn more about IDE Support for Vue in the
20     <a href="https://vuejs.org/guide/scaling-up/tooling.html#ide-support" target="_blank">
21       >Vue Docs Scaling up Guide</a>
22     >.
23   </p>
24   <p class="read-the-docs">Click on the Vite and Vue logos to learn more</p>
25 </template>
26 <script setup>
27   import { ref } from 'vue'
28
29   defineProps({
30     msg: String,
31   })
32
33   const count = ref(0)
34 </script>
35 <style scoped>
36   .read-the-docs {
37     color: #888;
38   }
39 </style>
40
41
42

```

En primeiro lugar o nome, **formado por dúas palabras** (o unha sola cunha máiscula no medio) e **iniciadas en maiúscula** (notación *PascalCase*) ou ben **separadas por un guión** *hello-word.vue* (notación *kebab-case*). Usarase a primeira opción.

En segundo lugar, analicemos a **estructura básica** dun compoñente.

Se vos aparece antes a zona de *script* ca de *template* non hai problema. Persoalmente, prefiro a forma ou orde tradicional. E dicir,

template => script => style.

A vantaxe da últimas versións de Vue e a inclusión de termo **setup** na declaración de *scripts*, que, como comentamos, ten as seguintes vantaxes que convén recordalas:

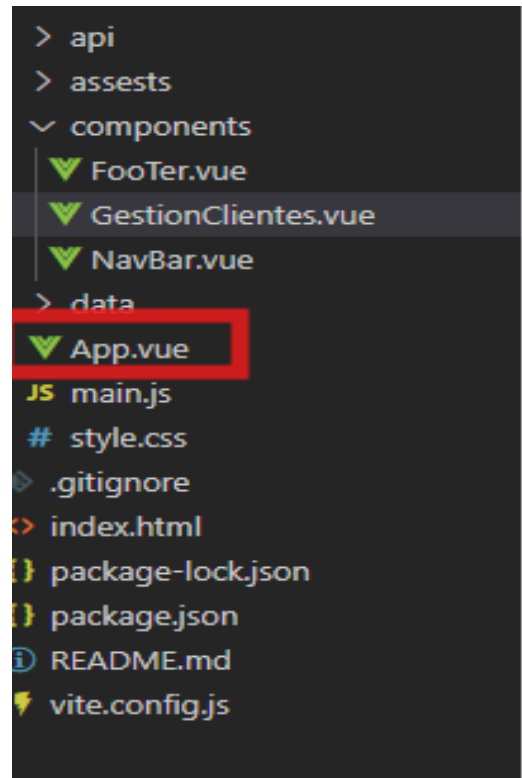
- código máis curto e claro, **mellorando a lexibilidade.**
- **importacións automáticas** e máis rendemento (Vue compila máis rápido) mellorando o rendemento.
- **non precisas** usar *export default* que pasa a *api*, nin *returns*, e o mellor tampouco *this*, que as veces da máis problemas que solucións.
- o uso de elementos *ref*, *reactive*, *computed*, *watch*, *onMounted*... entre outros, faise cunha sintaxes moito máis limpa e directa.

En resumo, e unha das cousas máis importantes é que todo o código está **automaticamente dispoñible no *template*** cando se carga a parte visual do compoñente.

Agora, botemos unha ollada ao compoñente ***App.vue*** ou **compoñente raíz**, e pai do resto dos compoñentes.

O primeiro que chama a atención é que esta situado fora do directorio ***components***. Isto é debido a dúas razóns básicas:

- é o **punto de entrada visual** onde se monta toda a *app*.
- está directamente conectado con ***main.js*** ou *main.ts*, que o monta dentro do ***index.html***.



Normalmente contén aspectos como o *router-view*, ou chamadas aos compoñentes *cabeceira*, *pé*, *menú* e outros elementos principais para **non ter que chamalos cada vez que se carga** un compoñente. Isto fai que o desenvolvemento da aplicación gañe en produtividade.



O funcionamento é básicamente o seguinte:

1. ***index.html***, o primeiro que se lanza nunha web,
2. carga ***main.js***
3. e este chama ***App.vue***

Unha última consideración, no navegador vemos o termo **Vite+Vue**, pero **Vite** non é algo exclusivo dun framework, neste caso de **Vue**.

Vite é unha **ferramenta de desenvolvemento frontend** pensada para traballar con aplicacións *web* actuais dun xeito **moi rápido e eficiente**, dito de forma sinxela **é o motor que se usa para crear, executar e construír proxectos web modernos**, especialmente con *frameworks* como **Vue, React ou Svelte**.

Vite encárgase de:

- Crear a estrutura inicial do proxecto
- Arrancar un **servidor de desenvolvemento**
- Recargar a aplicación ao instante cando hai cambios (**Hot Module Replacement**) e cargando só o código que precisa e como usa ES Modules nativos do navegador faina moi rápida.
- Xerar a versión final optimizada para produción
- Traballa con Vue 3, React, Vanilla, Svelte,....

A.5. Resumo

Nesta primeira unidade aprendemos a:

- Comprender que é **Vue.js**, un framework progresivo e lixeiro para crear aplicacións reactivas baseadas en compoñentes.
- Entender o concepto de **aplicación reactiva** e **SPA (Single Page Application)**.
- Coñecer a **estrutura básica dun proxecto Vue** xerado con **Vite**.
- Diferenciar as tres partes dun compoñente: *template*, *script (setup)* e *style*.
- Instalar o **entorno de desenvolvemento** completo:
- **VS Code** e as súas extensións máis útiles (*Volar, ESLint, Prettier...*).
- Como instalar **Node.js** e **npm/nvm**.
- Creación dun proxecto: ***npm create vite@latest nomeproxeto --template vue***.
- Analizar o funcionamento de compoñentes básicos como **HelloWorld.vue** e **App.vue**, e entender o seu papel na aplicación.
- Coñecer as vantaxes do novo **script setup**, que simplifica a sintaxe e mellora o rendemento.